Module Breakdown & Build Plan

ECITY — Mobile Shop Management Web App

Version 1.6 · 2026

15 modules, sequenced for one developer

Companion documents: PRD, Module Breakdown, Engineering Design, Deployment Guide, Database Guide

1. How to Read This Document

The application is split into **14 modules (M0–M13)**, ordered so that each one is buildable, testable and demonstrable on its own, and only depends on modules already finished. Build them in order. Do not start a module until its dependencies are marked done.

Each module lists:

- **Goal** — what the business can do once it ships
- **Delivers** — feature IDs from the PRD (which map to the source Feature List v3 section numbers)
- **Data model** — the tables introduced or extended
- **Screens** and **Server work** — the concrete surface to build
- **Done when** — acceptance criteria; if you cannot demonstrate all of them, the module is not done
- **Depends on** and **Effort** — a full-time-equivalent estimate for one experienced developer

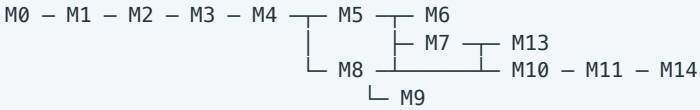
> **Estimates assume one developer working full time**, including tests and review. At roughly 20 productive hours a week, multiply by about two for a calendar estimate.

2. Delivery Map

#	Module	Effort (FTE weeks)	Depends on	Release
M0	Foundations: project, auth, RBAC, branch context, audit	2.0	—	Alpha
M1	Master data: business, branches, users, customers, suppliers	1.5	M0	Alpha
M2	Inventory core: products, branch stock, device units (IMEI)	2.5	M1	Alpha
M3	Purchases & supplier ledger	2.0	M2	Alpha
M4	Sales & billing & invoices	3.0	M3	Alpha
M5	Payments, credit sales & customer dues	1.5	M4	Alpha
M6	Returns, exchange & trade-in	2.0	M5	Beta
M7	Money: expenses, cash drawer, bank accounts, daily closing	2.5	M5	Beta
M8	Branch transfers & stock adjustments	2.0	M4	Beta
M9	Global search & IMEI device history	2.0	M8	Beta
M10	Dashboards & analytics	3.0	M7, M8	v1.0
M11	Reports, exports, imports & opening balances	2.0	M10	v1.0
M13	Notifications, alerts & warranty tracking	1.5	M7	v1.0
M14	Backup, data protection, hardening & go-live	1.5	all	v1.0
	Total	**~29.0 FTE weeks**		
~~M12~~	~~External billing system integration (Excel feed)~~ — **deferred, see §2.3**	~~3.5~~	M7, M11	On request

Add roughly 15% for discovery, rework and the open questions in PRD §11 — plan for **33–34 weeks full time**, or about **8 months at 20 hours a week**. M12 is not in that figure; building it later adds its 3.5 weeks.

2.1 Dependency shape



M6, M7 and M8 are independent of each other once M5 is done — if a second developer joins, that is the point to split.

2.3 The two businesses are separate. M12 is deferred.

Decided 2026-09-07. NEW handsets are bought, stocked and billed **entirely in the shop's other system**. ECITY handles **everything else** — USED, ER, ACT and GLOBAL. The two do not overlap, and nothing is imported between them.

M12 existed to reconcile a shop running both systems over the *same* stock. That is not what is happening: the stock is split, not shared. So M12 is not being built for v1.0, and ships only if the shop later decides it wants NEW sales visible in here too.

What already enforces the split, and stays. All of it came in earlier modules and none of it is removed:

- `business.new_stock_sales_channel`, defaulting to **EXTERNAL** — the line between the two businesses (OQ-11)
- `device_unit.sales_channel` (`ECITY` | `EXTERNAL` | `BOTH`), defaulted from that setting, overridable per device
- The till refuses an `EXTERNAL` device at search time and again at save time: *"This device is billed through the other system."* This is now a permanent rule, not a stopgap — it is what stops the two businesses crossing
- `SOLD_PENDING_IMPORT`, `source` (`ECITY` | `LEGACY`), and `daily_closing.external_feed_imported` / `override_reason` (M7), all dormant

What this means for what ECITY reports. ECITY is the used-and-refurb business, completely. Its stock, cash, dues, profit and analytics are the whole truth *about that business* — they are not, and are not meant to be, the whole truth about the shop. NEW is somewhere else and stays there.

The one thing to confirm operationally. If NEW sales take cash into the **same physical drawer**, the counted cash at closing will include money ECITY never billed, and every day will read as an overage. Two ways out, both fine: keep a separate drawer for NEW, or record the NEW takings as a single cash movement so the day balances. Worth settling before go-live, because it turns the daily closing from a control into noise. Nothing in the code assumes either answer.

If the shop ever changes its mind, M12's design is kept intact below and the hooks are already in the schema — it would start from that page rather than a blank one.

2.2 Rules that apply to every module

1. **Branch scope and permission are enforced on the server** in every endpoint you add. Never rely on the UI.
2. **Money is integer paise.** No floats, ever.
3. **Stock, money and device-status changes are written in the same database transaction as their document.**
4. **Nothing is hard-deleted.** Documents move to Cancelled / Voided / Reversed states.
5. **Every device-touching action appends a `device_event` row.** M9 is only possible because M2–M8 did this faithfully.
6. **State goes in the right tier.** Server data (inventory, sales, customers) is read by Server Components or TanStack Query — never copied into a client store. Shared browser-only state uses Zustand, and the only thing

that qualifies before M10 is the bill being built (M4). Everything else is `useState`. M0–M3 need no store at all.

7. Every module ends with: migrations committed, seed/demo data updated, tests green, and a five-minute demo of the "Done when" list.

M0 — Foundations

Goal. A running, deployed, secured skeleton: someone can log in, land in a branch context, and every action they take is audited.

Delivers. FR-1.1 – FR-1.5, FR-31.1, plus the technical base for everything else.

Data model. `business`, `user`, `role`, `permission`, `role_permission`, `user_branch`, `session`, `audit_log`.

Screens. Login, forgot/reset password, app shell (sidebar, header, branch switcher, user menu), user list, user create/edit with role and branch assignment, role editor, audit log viewer with filters.

Server work.

- Session-based authentication, password hashing, password reset tokens with expiry.
- A single authorisation helper used by every endpoint: `requirePermission(user, permission, branchId)`.
- Request-scoped branch context: the selected branch (or "all branches" for those permitted) resolved once and applied to every query.
- Audit writer invoked from a common data-access layer so that new features are audited by default rather than by remembering.
- Database migrations, seeding, error handling, structured logging, health check.
- CI: typecheck, lint, tests, build an image, push to GHCR, deploy over SSH. Provision the Hetzner server, Docker Compose stack (app, Postgres 16, pg-boss worker, Caddy) and the staging stack — follow the Deployment Guide.

Done when.

- A user can log in, reset a password and log out; sessions expire and can be revoked.
- An Admin can create a Manager restricted to Branch A, and that Manager cannot read Branch B data — verified by calling the API directly, not just by the hidden menu.
- Every create/update/delete writes an audit row with user, timestamp, entity and old/new values.
- ~~The app is deployed to staging from the main branch automatically, and the production stack serves HTTPS on the real domain.~~ **Deferred — see below.**

> **Deferred from M0: the staging deployment.** The CI and deploy workflows are written and the code is on GitHub, but no server has been provisioned. A server costs about ₹550/month and buys nothing until there is something worth showing the shop owner. **Provision it at the end of Alpha (after M5)**, when one branch can transact end to end and the owner can try it on real hardware. Until then, migrations are tested locally. This is a deliberate deferral, not a skipped criterion: M0 is otherwise complete, and this line moves to M5's "Done when".

Depends on. — **Effort.** 2.0 weeks.

M1 — Master Data

Goal. The shop is configured and the people it trades with exist.

Delivers. FR-2.1 – FR-2.4, FR-3.1, FR-3.2, FR-3.8, FR-5.10, FR-6.6, FR-10.1.

Data model. `branch`, `tax_rate`, `payment_method`, `expense_category`, `customer`, `supplier`, `attachment`.

Screens. Business profile & logo, tax configuration, payment methods, branch list / create / edit / deactivate, customer list + profile + create/edit, supplier list + profile + create/edit.

Server work. CRUD with validation; duplicate detection on customer phone and supplier phone/GST; deactivation semantics (a deactivated branch or supplier is hidden from new-transaction pickers but remains fully readable in history); attachment upload with signed URLs.

Done when.

- Business profile, tax rates and payment methods drive the pickers used later.
- Branches can be created, edited and deactivated; a deactivated branch cannot be chosen for a new transaction but its history is intact.
- Customer and supplier profiles exist with all PRD fields, are searchable by name/phone/email, and show an (empty for now) history tab.

Depends on. M0. **Effort.** 1.5 weeks.

M2 — Inventory Core

Goal. Stock exists, in branches, and every handset is an individually tracked device with an IMEI, a main type and a status. **This is the module that decides whether the rest of the system is correct** — take the extra day here.

Delivers. FR-4.1 – FR-4.12, FR-5.1 – FR-5.7 (the classification, identifier and status rules).

Data model.

- `category`, `brand`, `product` (accessory or mobile model), `branch_stock` (product × branch: qty, min_qty)
- `device_unit`: product_id, variant, ram, storage, colour, purchase_price, selling_price, tax, warranty, supplier_id, purchase_date, **main_type** (`NEW|USED|ER|ACT|GLOBAL`), **is_new_cut** + new-cut details, current_branch_id, status, `primary_imei` (cached for display only)
- `sales_channel` on `device_unit`: `ECITY` | `EXTERNAL` | `BOTH`, defaulted from the shop's `new_stock_sales_channel` setting — so a device belonging to the other business can never be sold here. Added now; retrofitting it after M4 means revisiting the billing screen. This is the line between the two businesses (§2.3), not a temporary guard
- `source` on `device_unit`, `purchase` and `sale`: `ECITY` | `LEGACY` — where the record came from. One column, added now, saves a migration later
- `device_identifier`: device_id, **value**, **type** (`IMEI` | `SERIAL`), slot, is_primary — **a device has one or more identifiers**. Never model these as `imei_1` / `imei_2` columns on the device; the whole point of the separate table is that a third identifier, or a laptop serial, costs nothing later
- `category.identifierType`: `IMEI` | `SERIAL` | `NONE` — what a serialised category's units are identified by
- `battery_health_percent` on `device_unit`: nullable 1–100, checked in the database. Blank for sealed new stock; it drives used and trade-in pricing
- `device_event` (append-only): device_id, seq, event_type, occurred_at, branch_id, from_branch_id, to_branch_id, ref_type, ref_id, actor_id, payload
- `stock_ledger` (append-only) for non-serialised quantity movement

Constraints to enforce in the database, not only in code.

- `is_new_cut = true` is permitted only when `main_type = 'GLOBAL'` (check constraint).

- `device_identifier.imei` unique across the entire business.
- Exactly one `is_primary` identifier per device (partial unique index).
- `sales_channel = 'EXTERNAL'` blocks the device from ECITY sale (enforced in M4).
- `branch_stock.qty >= 0`.

Screens. Product list with search and kind/inactive filters; product create/edit, with image upload on the edit page once the product exists; device list with filters for branch, main type, GLOBAL/NEW CUT, brand, category, supplier and status; device detail page (a static view now — M9 turns it into the full timeline); low-stock view.

Manual device entry. A small create-device form is also needed, for opening stock and corrections — this module's own acceptance criteria describe form behaviour (`imei_slots` showing one field or two), which nothing else provides until M3. It is deliberately secondary: labelled *Add manually*, and it states that supplier stock belongs in a purchase. The bulk path is M3's IMEI-capture grid.

Not phones only. The five main types and NEW CUT apply to every serialised item — laptops, MacBooks, speakers, tablets. What differs is the identifier: `category.identifierType` is `IMEI` for phones and `SERIAL` for other electronics, and it drives both the database format check and the label on the form. Accessories stay counted, with no identifier.

Server work. Stock read/write helpers that every later module calls (`increaseStock`, `decreaseStock`, `setDeviceStatus`), each of which writes the ledger/event row automatically. Get this API right once and M3–M8 become simple.

Device creation and update take `imeis: string[]`, never a pair of named fields, and every lookup resolves a device by *any* of its identifiers. Add the `imei_slots` business setting (default **1**) — it governs **how many input fields the UI renders and nothing else**. The API, importer, search and reports always accept and return the full list, so raising the setting later is a configuration change with no migration, no backfill and no code release.

Done when.

- A device can be created with each of the five main types, and a NEW CUT device can only be created under GLOBAL — rejected at the API and by the database for the other four.
- A device created through the API with three IMEIs stores all three, marks one primary, and is found by searching any of them; the same IMEI on a second device is rejected with a message naming the first.
- With `imei_slots = 1` the form shows exactly one IMEI field, and setting it to 2 in **Settings** → **Business** shows two with no deployment.
- A laptop or speaker registers against a serial number rather than an IMEI, carries the same five main types, and is refused a letter-bearing value where an IMEI is expected.
- A product's stock reads as a **number** in both cases: counted products sum their branch stock, serialised ones count the units still in stock. Two identical handsets show as 2.
- Low stock lists counted products at or below their branch minimum.
- The device list filters correctly by every filter in FR-4.6, including *normal GLOBAL* vs *GLOBAL + NEW CUT*.
- Accessory stock is per branch; the same product shows different quantities in two branches.
- Every status change writes a `device_event` row; no code path changes a device without one.

Depends on. M1. **Effort.** 2.5 weeks.

On PRD OQ-1 and OQ-2: neither blocks the start of this module. The five main types are stored as codes; what **ER** and **ACT** stand for is a display label in configuration, not a schema concern. Ask during M2 whether either carries a sub-designation the way GLOBAL carries NEW CUT — if one does, it is a nullable column plus a check constraint, the same shape as `is_new_cut`.

M3 — Purchases & Supplier Ledger

Goal. Stock legitimately enters a branch, and what is owed to suppliers is tracked.

Delivers. FR-5.8 – FR-5.15, FR-14.1 – FR-14.3.

Data model. `purchase`, `purchase_item`, `supplier_payment`, `supplier_ledger_entry`, purchase attachments.

Screens. Purchase list with filters; purchase entry form (supplier, branch, date, items) with a fast IMEI-capture grid for mobile lines — scan IMEI, pick main type, mark NEW CUT when GLOBAL. The grid renders `imei_slots` IMEI columns, which is **one column in v1**, while the request body it submits is always a list; purchase detail with attachments; supplier payment entry; supplier outstanding report; supplier profile history tab now populated.

Server work.

- Confirming a purchase, in one transaction: create device units for each IMEI, increase accessory stock, write `device_event: purchased/received`, post the supplier ledger entry, set payment status.
- Reversal: a purchase can be reversed only if none of its devices have moved on. If any have, the error must name the blocking IMEIs.
- Purchases record their `source` (`ECITY` by default), so stock bought through the other system can later be imported without ambiguity.
- Duplicate IMEI detection at entry time, checked against **every** identifier of every device in the business, with a clear message naming the conflicting device.

Done when.

- Confirming a purchase with 5 mobiles and 20 accessories raises branch stock correctly and registers 5 device units with the right main types, each with its IMEI list stored and a primary identifier set.
- Partially paying a purchase leaves the correct supplier outstanding, and the supplier profile shows the purchase, the payment and the balance.
- Reversing an untouched purchase removes the stock and the ledger effect and leaves an audit trail; reversing a purchase whose device was sold is refused with a specific reason.

Built as (revised after M11): a purchase line carries the specs, and stamps them on every handset it creates. As first built, a purchase knew only the product, the IMEI and the main type. So ten iPhone 17s came in as ten handsets that did not know they were 256GB green, and the specs had to be typed in afterwards, one device screen at a time — the shop found this the moment a real shipment arrived.

The fix is not product-level specs. "iPhone 17 256GB Green" as its own product means four storages × six colours = **24 products for one model**, none of which exist until the stock walks in, so product creation moves into the purchase flow anyway and the catalogue fills with near-duplicates that two people spell differently. It would not even finish the job: battery health, grade and IMEI stay per-handset regardless. And for used stock it is simply the wrong shape — two 256GB green handsets at 82% and 94% battery are not the same thing.

So the specs sit on the **line**, which is what the line already was: its unit cost is a single number, and a 256GB does not cost what a 128GB costs, so a mixed-spec line was never possible. The line fills RAM, storage, colour and variant onto every unit; any single unit can override them, plus its own battery health, for the one piece in the batch that is not like the others. **Duplicate line** copies the product, cost and specs while clearing the identifiers, so the second combination in a shipment is two edits rather than a second full entry. Battery health has no line default on purpose: on used stock it differs on every piece, and a default would be a number somebody trusted.

They stamp at creation and stop there. Correcting a line afterwards does not rewrite handsets already booked in — the same rule an invoice follows (docs/03 §4.9); a handset is corrected on its own screen, where it always could be.

Depends on. M2. **Effort.** 2.0 weeks.

M4 — Sales & Billing

Goal. The counter can sell, and produce an invoice. The highest-traffic screen in the product.

Delivers. FR-6.1 – FR-6.8, FR-26.1, FR-26.3, FR-26.4, FR-26.5, FR-38.2, FR-38.3.

Data model. `sale`, `sale_item`, `sale_payment`.

Built as: no separate `invoice_series` table was needed. M3 already introduced `document_sequence`, which allocates gapless per-branch, per-year counters under `SELECT … FOR UPDATE`; invoices reuse it with a different document kind. One table, one concurrency proof, one place to audit.

Screens. The billing screen — a single keyboard-driven page: search by name/SKU/barcode/IMEI, cart with line discounts and tax, customer attach/create inline, split payment across methods, save & print. Sale list with filters. Sale detail. Printable invoice in A4 and 80 mm thermal layouts, plus PDF download and share. Customer history (FR-6.7) on the customer record: spend, purchases across every branch, and what is still owed.

Built as: the customer, supplier and product fields are **searchable pickers**, not `<select>`s. A capped dropdown silently omits records — found in production-shaped data at 571 suppliers and 693 serialised products against a 500 cap — and the omission is invisible to the user.

Client state — the first module that needs a store. The bill being built is real client state: cart lines, per-line and bill-level discounts, the attached customer, split payments across methods, and later the trade-in from M6. Four or five sibling components read and write it — the search box, the cart, the totals panel, the payment panel — which is past what props or context handle cleanly.

- Use **Zustand** (`npm i zustand`), one store scoped to the billing screen. Not Redux: same result, a quarter of the code.
- **Persist the store to** `localStorage`, including the bill's idempotency key. This is what satisfies PRD §9.3 — a dropped connection or an accidental refresh must not lose a half-built bill, and re-submitting must not create a second one.
- Clear the store only on a confirmed save, never optimistically.
- **TanStack Query** joins here too, for client-side reads (product and IMEI search as the user types). Keep the division strict: Query owns anything Postgres owns; Zustand owns only what exists in the browser. Never copy sale or stock data into the store.

Built as: Zustand with `persist` as planned. TanStack Query was **deferred** — the search box is the only client-side read in M4, and a debounced `fetch` covers it without adding a second data layer. Revisit when a screen needs shared caching or background refetch.

Server work.

- Availability check and device lock at save: the exact device must be `In Stock` at the selling branch, and a conditional update prevents two tills selling the same IMEI.
- **Channel check:** a device with `sales_channel = 'EXTERNAL'` is refused at search time and at save time, with the message *"This device is billed through the other system."* NEW stock belongs to the other business entirely (§2.3), so this is a permanent rule rather than a stopgap — it is what keeps the two from crossing.
- **Mark as sold externally**, a permissioned action setting `SOLD_PENDING_IMPORT`: stock drops now. The fallback for a `BOTH`-channel device — rare, since the two businesses are separated (§2.3). Stock ends up right; the other system's invoice number is not recorded here unless someone types it.
- Tax computation honouring inclusive/exclusive configuration.

Built as: **OQ-4 was answered "yes — statutory".** So the tax engine also splits every line into CGST/SGST (intra-state) or IGST (inter-state), and the invoice carries HSN/SAC per line, the place of supply, and an HSN-wise summary. Place of supply follows the customer's registered state and falls back to the branch's; where neither is known the sale stays intra-state rather than guessing IGST. The split is stored, not derived at print time, because a reprint years later must be identical even if the branch or customer has since moved state. CGST takes the floor of the halving and SGST the remainder, so the two always add back to exactly the tax charged.

Built as: **OQ-7 was answered "no — a reliable connection is acceptable".** The persisted cart still covers a refresh or a dropped connection, and idempotency still prevents a double bill, but saving requires the server. Offline-first would be its own module.

- Invoice number allocation, gapless per branch series, safe under concurrency.
- Idempotent sale submission keyed by a client-generated id, so a retried request after a dropped connection does not create a second bill.
- Sale completion in one transaction: stock down, `device_event: sold`, payments posted, invoice numbered.

Done when.

- An accessory-only cash sale completes in under 20 seconds using only the keyboard and scanner.
- A mobile sale shows the main type on the line, and NEW CUT detail for a GLOBAL device.
- Selling an IMEI that another user has just sold fails clearly rather than double-selling; the device's status is `Sold` exactly once.
- A NEW device cannot be added to a bill at all, and the refusal explains why.
- Both invoice formats print correctly, and the branch invoice series has no gaps or duplicates after 200 concurrent test sales.
- A half-built bill survives a browser refresh and a dropped connection, and submitting it twice creates exactly one sale.

Depends on. M3. **Effort.** 3.0 weeks. *OQ-4 and OQ-7 both answered during M4 — see below.*

M5 — Payments, Credit Sales & Customer Dues

Goal. Credit is controlled: what is owed, by whom, since when, and where it was collected.

Delivers. FR-7.1 – FR-7.6.

Data model. `customer_payment`, `customer_payment_allocation`, `customer_ledger_entry`, credit fields on `sale` (due_date, credit_notes).

Built as: there is deliberately **no stored** `outstanding` **column**. It is always summed from the append-only ledger (docs/03 §4.2), so the screen and the books cannot disagree. The customer side mirrors the supplier side table for table, which means one mental model and one place to fix allocation or voiding. `customer_ledger_entry` is append-only, enforced by a database trigger rather than by convention.

Screens. Payment collection against one or many open sales; customer dues list with aging; customer statement; overdue view; payment receipt print.

Server work. Ledger-based balance, never a stored mutable number: outstanding is always derived and reconcilable. Allocation of a payment across invoices. Automatic status flip to Paid when settled. Both the sale branch and the collection branch recorded on the payment.

Built as:

- **One definition of "what a sale has been paid".** Counter payments (`sale_payment`) and later collections (`customer_payment_allocation`) are summed by a single shared SQL expression used by the sale detail, the sale list, the payment-status filter and the customer history. Before M5 there were four separate copies; leaving them would have meant a bill settled by a receipt still showing UNPAID in the list.
- **Receipts are gaplessly numbered** through the same `document_sequence` machinery as invoices, per-branch where the branch has its own prefix.
- **A voided receipt settles nothing** — the allocations stay as a record of what the receipt claimed, and the ledger gets a REVERSAL entry rather than an edit. The invoice reopens automatically.
- `customer_payment.void` **is its own permission.** Counter staff may collect (a customer clearing their tab is the counter's job) but may not erase a collection already recorded.
- **Allocation runs inside the caller's transaction.** The first cut queried the pool from inside an open transaction and deadlocked under concurrency — 25 parallel receipts hung for 70 seconds. It also did two queries per open invoice; it is now one.
- **The agreed terms appear on the bill.** FR-7.2 says a credit sale stores its due date and notes; storing them without showing them would mean the salesperson agreed a date nobody could see again. An unpaid sale carries an outstanding figure, the due date (in red once past), the note, and a link straight to collection.
- **Branch-wise (FR-7.6) reports two different figures deliberately.** A branch is shown what it is *owed* (on bills it raised) beside what it *collected* (money taken at its counter, whoever raised the bill). They are not meant to match: a customer may buy at one shop and settle at another, and a branch doing the group's collecting should look like it. Collections are bounded by a date range, defaulting to the current month, because "collected" with no period is not a number anyone can act on.

Done when.

- A credit sale creates the correct outstanding and due date; three partial payments settle it and flip it to Paid.
- A payment taken at Branch B against a sale made at Branch A records both branches and appears in both branches' cash/collection figures correctly.
- Customer outstanding recomputed from the ledger equals the displayed balance for every customer in the test data.
- Aging buckets 0–7 / 8–30 / 31–60 / 60+ are correct against hand-checked dates.
- **Carried from M0:** the Hetzner server is provisioned, and pushing to `main` deploys to staging automatically. Alpha is the first release someone outside the project sees, so it needs somewhere to live. See the Deployment Guide.

Built as: the pipeline is written, and the deployment guide §6.2 has the half that needs a provider account. **The provider changed:** Hetzner's Singapore region never sold the CX22 this plan assumed (the CX line is EU-only), and their prices rose 144–190% on 15 June 2026, which made them both the dearest option and the furthest away. The recommendation is now an India region — Vultr Mumbai or DigitalOcean Bangalore — which is cheaper, ~20 ms instead of ~150 ms, and keeps the shop's records in India. Nothing in the pipeline was provider-specific, so no code changed; see deployment guide §1.1. Fixing the workflow found three faults that would each have broken a real deploy: it did not wait for CI (`needs: []`), the migration step ran `drizzle-kit` from an image that does not contain it with `|| true` swallowing the failure, and the compose file referenced a `worker.js` that does not exist. It also now runs `db:sync-roles`, without which a module that adds a permission ships a screen nobody can open.

Depends on. M4. **Effort.** 1.5 weeks.

M6 — Returns, Exchange & Trade-In

Goal. Goods come back and old phones come in, without corrupting stock, money or device history.

Delivers. FR-8.1 – FR-8.5, FR-9.1 – FR-9.3.

Data model. `sales_return`, `return_item`, `refund`, `trade_in`, device inspection fields.

Screens. Return by invoice / customer / IMEI; full, partial and exchange return flows; inspection queue for returned devices with the classify action (Available / Used / Damaged / Repair Required); trade-in capture inside the billing screen, with valuation and difference payable — extends M4's Zustand cart store rather than introducing a second one.

Carried in — correcting a device (raised during M5). There is no way to edit a device after it is created, which means a wrong main type is permanent. That matters more than it sounds: main type decides whether a handset reaches the till at all, so one keystroke can hide sellable stock for good, with no way back. This module already builds a reclassify action for returned devices, so the general edit belongs here rather than in a module of its own.

- **Edit a device** (manager and admin): main type and NEW CUT, variant / RAM / storage / colour, battery health, purchase and selling price, tax rate, supplier, warranty, and sales channel.
- Every change writes an audit entry **and** a `device_event`, so the M9 timeline shows corrections as part of the device's history rather than silently rewriting it.
- Identifiers stay out of the edit form. Changing an IMEI is not a correction, it is a different handset; the existing uniqueness and history rules make that deliberate.

Server work.

- Returned mobiles go to `Returned / Inspection`, never straight back to sellable (this is the rule most likely to be got wrong).
- Refund posts against a payment method and the branch cash drawer or account.
- Trade-in devices are created as new device units in the receiving branch with the correct main type and GLOBAL/NEW CUT information, and their own `device_event` chain begins.
- Main type and NEW CUT survive the whole return path unchanged.

Built as:

- **Condition is a separate field from main type.** FR-8.3 wants grading into Available / Used / Damaged / Repair Required, and FR-8.4 wants main type preserved. Overloading `mainType` with condition would fail the acceptance test outright: a returned GLOBAL + NEW CUT handset graded "used" must still read GLOBAL. So `device_unit.inspection_grade` is its own column. Condition and classification are different facts about the same handset, and changing the classification is the separate, audited device edit.
- **The status transition table already enforced the rule.** M2 wrote `SOLD → RETURNED` and `RETURNED → IN_STOCK | DAMAGED | REPAIR | LOST`, so a returned handset physically cannot reach sellable without passing through inspection. M6 only had to use it.
- **A refund and the customer ledger move in opposite directions.** Returning goods always reduces what the customer owes by the value returned; paying the money out adds it back, because they received it. Doing only the first while also handing over cash is the classic returns bug and the shop pays twice. Both directions are tested.
- `sale_item.device_id` **stopped being unique.** M4 made it unique so a handset could not be billed twice, which was right until returns existed — a resold device is legitimately a second sale line. Double-selling is still prevented by the conditional `expectedStatus = IN_STOCK` update, which is transactional and covered by M4's two-tills test.
- **A trade-in is not a discount.** The invoice shows the full price of what was sold and GST is charged on that; the agreed value settles part of the bill, exactly like a payment. Putting it in the tax calculation would understate the GST due.
- **So the trade-in is counted by the one expression that answers "what has this bill been settled by".** M5 unified counter payments and later collections into `saleReceivedSql()`; the agreed value of a linked trade-in joins them there. Settling it anywhere else would have let the sale list, the dues screen, the aging buckets and

the customer statement disagree about the same exchange. The bill's own `total_paise` and its GST never move.

- **A refund is capped by what the bill was actually settled by.** Goods bought on credit and returned before a rupee was paid would otherwise pay out cash the shop never received — the customer keeps the debt and takes the money. The account reversal still clears the full value of the goods, so a part-paid bill hands back exactly what was paid and writes off the rest. A credit note is not capped: crediting the account is the reversal, not a payout.
- **The trade-in is locked for the length of the sale transaction.** `select ... for update` with a `sale_id is null` check, so two tills cannot put the same handset against two bills — which would settle its value twice and cost the shop the difference.

Done when.

- A returned GLOBAL + NEW CUT device still reads as GLOBAL + NEW CUT after return and reclassification.
- A returned device is not sellable until an authorised user classifies it Available.
- An exchange sale records the trade-in value, the difference paid, the new device sold and the old device received, all linked to one another.

Depends on. M5. **Effort.** 2.0 weeks.

Also delivered here — the GST registration switch (FR-2.6, FR-26.6). Asked for on 2026-09-07: the shop sells mostly used handsets, trades below the registration threshold, and expects to register within about a year. Built alongside M6 rather than as its own module because it is one setting and a set of conditionals, not a subsystem.

Built as:

- **Two flags, not one.** `business.gst_enabled` is what the shop is *today* and drives the forms and the till. `sale.gst_enabled` is what the shop *was* when that bill was issued and drives that invoice for good. The setting moves; history does not.
- **Why the second one is not optional.** Nothing archives the invoice PDF — `/api/sales/[id]/pdf` renders from the rows on every request. So a reprint is always a fresh render, and if the invoice asked the business setting whether to print GST, registering next year would reprint every bill from this year as a tax invoice, headed with a GSTIN the shop did not have at the time. Bills get reprinted for warranties, returns and disputes; M6 itself takes returns against old bills.
- **Zero tax cannot stand in for the flag.** Exempt and zero-rated goods sold *under* GST are also zero, and those do belong on a tax invoice with an HSN. "Tax was zero" and "there was no GST regime" are different facts that produce the same number, so the number cannot carry it.
- **The rate is forced server-side, not just hidden.** `createSale` reads the setting and zeroes the rate itself, so a stale browser tab still holding a tax rate cannot put tax on a bill. Place of supply, the state code and the HSN snapshot are left null for the same reason — they are GST concepts with nothing to say outside one.
- **The document is retitled, not just stripped.** An unregistered dealer must not issue something headed "Tax Invoice" or carrying a GSTIN. Leaving the heading while removing the tax would produce a document that misstates what it is.
- **Nothing is deleted.** The GST columns, the tax rates and `src/lib/gst.ts` all stay in place, dormant. Registering is flipping the switch.
- **Purchases need no flag.** A purchase record is not a document the shop issues to anyone — the supplier provides theirs — so those screens read the live setting. Worth knowing operationally: while unregistered, tax a supplier charges is not reclaimable, so enter the full price paid as the unit cost and leave the tax box empty, and margins read correctly.

Done when.

- With GST off, a bill charges no tax even if the request carries a tax rate, and the invoice has no "Tax Invoice" heading, GSTIN, HSN column, tax column or HSN summary.

- Turning GST on afterwards leaves every bill issued before it printing exactly as it was issued.
- Turning GST off does not strip GST from bills already issued under it.

M7 — Expenses, Cash Drawer, Bank Accounts & Daily Closing

Carried in — correcting a purchase (raised during M5). A confirmed purchase can be reversed but not edited, so a typo in an invoice number or date has no fix once a unit from it has sold, because reversal is then refused.

- **Edit a purchase's metadata** (manager and admin): supplier invoice number, purchase date, notes.
- **Lines, quantities and costs stay uneditable.** A confirmed purchase has already moved stock, created device units and posted to the supplier ledger; editing a cost afterwards would leave the ledger disagreeing with the stock and nothing to reconcile against. Reversal already handles that case and already refuses once a unit is sold — a working reversal is worth more than an edit that quietly corrupts the books.
- Belongs here because this is the module that owns money movement and reconciliation.

Goal. The day balances. Expected money is compared with counted money, per branch, every day.

Delivers. FR-10.1 – FR-10.3, FR-11.1 – FR-11.5, FR-12.1 – FR-12.4, FR-13.1 – FR-13.5.

Data model. `expense`, `cash_drawer_day`, `cash_movement`, `account`, `account_transaction`, `daily_closing`.

Screens. Expense entry and list with receipt upload; cash drawer day view showing opening, every cash in/out, expected cash; accounts list with balances, receipts, payments, transfers and reconciliation; daily closing screen with the day summary, expected vs actual per method, computed difference, and confirm-close; closing history.

Server work.

- Every cash-affecting event from M4–M6 (cash sales, credit collections, refunds, expenses, supplier payments) posts a `cash_movement` into the branch's open drawer day. Backfill this into the earlier modules as part of this module's work.
- `Expected Cash = Opening + Cash In − Cash Out`, derived from movements, not stored.
- Closing a day freezes it; editing anything dated inside a closed day requires an explicit permission and is audited.
- **Expected cash covers what ECITY billed, and only that.** The other business's sales are not in here (§2.3), so if both take cash into the *same physical drawer* the counted figure will exceed what ECITY expects, every day. Settle that operationally — a separate drawer, or one cash movement recording the other business's takings. The closing still carries `external_feed_imported` and `override_reason` from the original M12 design, unused, so a gate could be hung on them without a migration.
- Opening balance of the next day carries from the previous day's actual count.

Built as:

- **The drawer opens itself.** A branch's day is created on its first cash movement, not by anyone pressing a button. A counter that starts selling has a drawer whether or not someone remembered to open one — which is the difference between a reconciliation that works and one that is missing the first hour of trade.
- **One place decides cash from non-cash.** `postByPaymentMethod` reads the `affects_cash_drawer` flag M1 set, so cash goes to the till and everything else to an account. Two callers cannot route the same method differently.
- **Expected cash is derived every time it is asked for.** Opening plus the sum of `cash_movement`, which is append-only with a database trigger. A stored counter would eventually disagree with the rows it claims to summarise, and a reconciliation that cannot be proved from its movements is worth nothing.

- **The backfill was the real work.** M4 cash sales, M5 collections and supplier payments, M6 refunds and M7 expenses all post into the drawer. A till that only knew about some of the day's cash would reconcile to nothing, so this had to go back through every earlier module rather than start from today.
- **A closing is stamped, never rewritten (OQ-5).** Expected, counted and difference are frozen at the moment someone signs off. A later correction is a new entry in that day, needs `closing.correct`, and is audited; the day's reports pick it up while the signed figures do not move, and the screen says so in as many words. Letting an owner edit a closed day was rejected because it is the straightforward way to hide a till shortage: come up short today, adjust yesterday, the difference disappears.
- **Reopening exists for the mistake people actually make** — closing at six and then taking a sale at seven. `closing.void` reopens the day, audited with a reason, but is refused once a later day for that branch has been closed. The voided closing is kept: that a day was closed and reopened is part of the record.
- **Tomorrow opens with what was counted, not what was expected.** The drawer starts with the cash physically in it, so a shortage does not silently repeat itself every morning.
- **Staff do not record expenses.** They see the drawer and the expenses that explain it, but booking money out of the till they are counting is the same hole as an editable closing.
- **An expense is voided, never deleted.** The row stays and the money is posted back, so the day it belongs to still tells the truth.
- **Reconciling an account records the statement; it does not move money.** If the books and the statement disagree, that difference is real and wants an explanation — an automatic adjustment would erase the only evidence that something was wrong. Correcting it is a visible `ADJUSTMENT` in the ledger.
- **A transfer writes both legs together.** One transaction, one `transfer_group`, so money cannot leave one account without reaching the other.
- **Carried in — correcting a purchase.** Built as specified: supplier bill number, date and notes only. The dialog says why the costs are absent.
- **An expense has its own page, because FR-10.2 wants a receipt on it.** The attachment plumbing came from M1; this is where it earns its place — the bill that explains a payment belongs with the payment, not in a folder.
- **An account has its own page too.** FR-12.3 asks for movements and a running balance, and a derived balance nobody can see the workings of is just a number to argue with.

Done when.

- A day with cash sales, a UPI sale, a credit collection, a refund, an expense and a supplier payment produces the correct expected cash, and entering a counted amount ₹200 short shows a ₹200 shortage attributed to that branch, user and timestamp.
- Two branches close independently and the consolidated reconciliation report adds up.
- A staff user cannot alter a transaction inside a closed day; an Admin can, and it is audited.
- A correction posted into a closed day leaves the signed expected, counted and difference untouched, and the day is marked as corrected after close.

Depends on. M5. **Effort.** 2.5 weeks. *OQ-5 answered — see the PRD.*

M8 — Branch Transfers & Stock Adjustments

Goal. Stock moves between branches under control, and physical/system differences are corrected on the record instead of silently.

Delivers. FR-3.6, FR-3.7, FR-28.1 – FR-28.3.

Data model. `stock_transfer`, `transfer_item`, `stock_adjustment`.

Screens. Transfer request (choose source, destination, devices by IMEI and/or accessory quantities); approval queue; dispatch (mark In Transit); receiving screen at the destination that scans IMEIs and flags anything missing or unexpected; transfer history; stock adjustment entry with reason code and evidence.

Server work.

- The state machine **Requested → Approved → In Transit → Received**, with Cancelled available up to receipt, enforced server-side; illegal transitions rejected.
- In-transit devices belong to neither branch's sellable stock — they must not appear as available anywhere.
- Receipt moves the exact IMEI to the destination branch and writes `device_event: transferred_out` / `transferred_in` with both branch ids and the date.
- Adjustments record branch, product or IMEI, main type, GLOBAL/NEW CUT, reason, user and timestamp, and require permission.

Built as:

- **M2 had already built most of it.** `IN_TRANSIT` was in the device status enum, `IN_STOCK → IN_TRANSIT → IN_STOCK` was in the transition table, and `TRANSFERRED_OUT` / `TRANSFERRED_IN` were in the event types. M8's job was to use them, not invent them.
- **Dispatch is where stock stops being sellable; receipt is where it starts again.** Nothing is sellable in between — that is the whole module. A handset sellable at both ends of its journey gets sold twice, and the second customer finds out afterwards.
- **The lifecycle is a transition table, not scattered `if`s** — the same shape as M2's device table, so it can be read and tested in one place. Skipping a step is refused: stock cannot leave before someone approved it.
- **A handset cannot be on two open transfers at once.** Checked when the transfer is requested, because two open requests for one IMEI means the second fails at dispatch — later, and more confusingly.
- **Cancelling in transit puts the goods back at the source.** They never reached the destination, so that is where they are.
- **Something that did not arrive is marked LOST, not left in transit.** A handset stranded `IN_TRANSIT` on a closed transfer sits in a state nothing can move it out of; something that left one branch and reached no other has to be accounted for.
- **The receiving screen scans, and starts with nothing ticked.** A box you scan into is safer than one you untick: if the scanner misses a handset the transfer reads short and someone goes looking, whereas starting ticked would let a missed scan pass as received. A handset that will not scan can still be ticked by hand. Accessories keep their sent quantity — there is nothing to scan.
- **An IMEI that is not on the transfer is flagged and recorded, never acted on.** Something physically here that the system thinks is elsewhere is a real problem, but moving it automatically would be inventing a transfer nobody authorised. It goes onto the transfer's discrepancy note so a person finds out where it came from.
- **An adjustment carries evidence** — a photo of the damage, or the count sheet — on its own page, using M1's attachment plumbing.
- **A transfer event names both branches in its columns, not its payload.** `setDeviceStatus` had been overloading one field to mean two things — where the device now is, and where the event says it is going — so `device_event.from_branch_id` was never populated and a `TRANSFERRED_OUT` recorded the *sending* branch as its destination. The journey is now separate from the move: a dispatched handset stays with the sender while its event says where it is headed. M9's IMEI timeline reads these columns, so "moved from A to B" has to be answerable without parsing JSON.
- **A cancelled transfer claims no journey.** The return event has no from-branch, because the handset never reached the destination and saying otherwise would put it somewhere it was never at. A handset lost in transit is the mirror image: a from-branch and no destination.

- **The approval queue and the receiving screen are the list, filtered.** They are one click from the transfer list rather than two more pages that could drift apart from it.
- **Each step belongs to one end of the journey.** Approving and dispatching are the sending branch's; signing for a delivery is the receiving branch's. A receipt is someone at the far end confirming the goods turned up — if the branch that packed the box could attest they arrived, the confirmation would be worth nothing. Enforced server-side and mirrored in the UI, which tells you whose step it is instead of offering a button that would be refused. A user who can see every branch is exempt: they are the owner, and there is nobody else to check them.
- **An adjustment is never edited.** A wrong one is corrected by a second, so both stay visible — the same rule as every ledger.
- **A handset can only be damaged or lost, never "miscounted".** A miscount is about a quantity; a wrong record about one handset is a device correction (M6's edit). The form says so rather than silently allowing a meaningless adjustment.
- **The classification is snapshotted onto the adjustment** (FR-28.2), so a later reclassification cannot rewrite what it said at the time — the same principle as docs/03 §4.9.
- **Adjustments write to the stock ledger**, so they show up in FR-21's movement report rather than only in their own list.

Done when.

- A device transferred A → B cannot be sold at A once dispatched, cannot be sold at B until received, and after receipt carries a movement history showing both branches with dates.
- A cancelled in-transit transfer returns the devices to the source branch cleanly.
- A miscount adjustment changes accessory stock and appears in the audit log and in the inventory movement report.

Depends on. M4. **Effort.** 2.0 weeks.

M9 — Global Search & IMEI Device History

Goal. The flagship feature: one search box, everywhere, and an IMEI that opens a device's whole life.

Delivers. FR-30.1 – FR-30.7.

Data model. No new business tables — this module *reads* `device_event` and the documents it points to. Trigram indexes over products, customers, suppliers, invoices, SKUs, barcodes and IMEIs.

Deviation: the plan called for a **materialised search view**. It was not built. Twelve trigram indexes hit the targets on their own — search returns well inside 500 ms and a fifty-event history assembles in a fraction of the 1.5 s budget — and a materialised view would add a refresh to keep current, a staleness window, and one more thing to go wrong at a shop with a few thousand rows. Revisit it if the data grows enough to need it; the queries would not have to change.

Screens. A command-palette style global search available on every screen (keyboard shortcut), with results grouped by type — Devices, Products, Customers, Suppliers, Invoices — and a dedicated **Device History** page: identity header, commercial summary, current position, and a vertical timeline `Purchase → Seller → Branch → Transfers → Sale → Customer → Return/Repair/Other`, each entry linking to its source document.

Server work.

- One search endpoint that detects the input shape (15-digit IMEI, phone number, email, invoice number, free text) and routes accordingly, always filtered by the user's branch permissions. IMEI matching runs against

`device_identifier`, so **any** of a device's identifiers finds it.

- Device history assembly: one query over `device_event` ordered by sequence, joined to purchases, transfers, sales, returns and adjustments.
- Indexes to hit the < 500 ms and < 1.5 s targets in PRD §9.1.

Built as:

- **One endpoint that reads the shape of what was typed.** A 15-digit number is an identifier, ten digits is a phone, something with an `@` is an email, `INV/…` is a document. Ten digits is *also* a partial IMEI, so both are searched — guessing wrong there means a counter assistant with a customer in front of them gets nothing.
- **An unambiguous match skips the list.** A complete IMEI, or an invoice number matching exactly one document, navigates straight there rather than rendering a list of one for someone to click.
- **Matching runs against `device_identifier`, never the cached primary.** FR-30.5 wants any of a handset's identifiers to reach it, and a customer reads out whichever number is printed nearest — not necessarily slot 1.
- **Search is where an application leaks.** It touches every table at once, so every branch of it re-applies the caller's branch scope and the permission for the kind of record it is about to return. No blanket permission on the endpoint: that would either lock out staff who legitimately search, or hand them rows they cannot open.
- **Customer visibility follows the trade.** FR-6.7 makes the customer record business-wide, but FR-30.7 says a branch-limited user must not pull up someone from a branch they cannot see. Both hold if a customer is reachable when they have bought at a visible branch, or have not bought anywhere yet and so belong to nobody in particular.
- **The timeline resolves documents in one query per type, not one per event.** Fifty events pointing at four documents is four queries. A round trip per event is exactly how the 1.5 s target in PRD §9.1 gets missed for no reason.
- **Every entry is a sentence, not a label.** "Sold on INV/MAIN/2026/000123 to Anil" rather than "SOLD", and a reclassification names the fields that moved — "Reclassified" alone tells nobody anything, and correcting a main type is precisely what someone will later want explained.
- **Twelve trigram indexes.** Every `ilike '%term%'` in the search has one: a leading wildcard cannot use a btree index at all, so without them the box sequential-scans five tables on every keystroke.
- **The device page answers FR-30.5's table, not a summary of it.** Commercials means cost, sold-for, the discount on *that line*, the payment status and paid-versus-credit — read through the same `saleReceivedSql()` the sale list uses, so the two cannot disagree about whether a handset was paid for. Current position means status, branch, and the branch before, derived from the last move in the event log rather than a column that could only ever hold one.
- **The palette does not re-filter what the server returned.** `cmdk` would otherwise hide a customer who matched on a phone number the row does not display.
- **The header control is "Find anything", not "Search".** Every list in the application has its own Search button that filters *that list*; this one finds anything, anywhere. Sharing the word made the two indistinguishable — to a screen reader as much as to a test.
- **Two M7 tests only worked as a pair, in order.** One closed the day, the next reopened it; when the first failed, the day stayed CLOSED, and a closed drawer refuses cash — so the *next spec's* billing broke with no hint as to why. Each now sets up its own state.
- **And the first attempt at that fix made it worse, which is worth recording.** Pinning a branch before closing looked tidy, but the active branch is per *session* and every test signs in afresh: the first test closed Main while the second, starting from the default again, reopened a different branch. Main then stayed closed for the whole run. The fix is to pin nothing and let both tests act on the same default — state that is per-session cannot be set up in one test and relied on in the next.
- **A test run that crossed midnight found a real bug.** Four browser-side date defaults used `toISOString()`, which is UTC — a different day from the shop's between 00:00 and 05:30 IST. The expense form defaulted to

yesterday and capped its picker there, so during those hours nobody could record that evening's expense; the purchase form, a purchase's edit date and a bill's credit terms slipped the same way. `src/lib/date.ts` now holds one definition of the shop's day that both the server and every form use. Nothing to do with M9 — it was simply the first run that happened to be going at midnight.

- **M2's placeholder timeline dumped the event payload as JSON, and two tests had come to rely on it** — one asserting a raw status enum, one counting how many times a colour appeared. Replacing the dump with readable sentences broke both. They were updated to assert what a person sees, which is what they always meant.

Done when.

- Searching a full IMEI opens the device page directly; searching a partial IMEI lists candidates; searching the second or third IMEI of a multi-IMEI device opens the same page as its primary one.
- For a device that was purchased, transferred twice, sold on credit, returned and reclassified, the timeline shows all seven stages with dates and working links, and the identity header shows main type, NEW CUT status and every IMEI on the device with the primary one marked.
- A Branch-A-only user searching a Branch B customer's phone number gets no results, verified at the API.
- Device history for a device with 50 events renders in under 1.5 s.

Depends on. M8 (so that every event type exists to display). **Effort.** 2.0 weeks.

M10 — Dashboards & Analytics

Goal. The numbers. Branch dashboards, the owner's consolidated view, and the nine analytics areas.

Delivers. FR-15.1 – FR-15.3, FR-16 – FR-24, FR-35.1 – FR-35.3, FR-36.1 – FR-36.4.

Data model. Reporting views / summary tables. Consider nightly-refreshed rollups per branch per day (sales, cost, profit, units, payment mix) so that year-range queries stay fast; today's figures read live.

Screens. Branch dashboard; owner consolidated dashboard with branch comparison; and analytics pages for sales, product, brand, customer, credit, inventory, profit, payment and supplier — each with date range, branch selector, comparison mode, chart plus table, and drill-through.

Server work.

- One analytics query layer with a consistent shape: date range, branch set, grouping, comparison period.
- The mobile-type dimension is mandatory across product, inventory, profit and insights analytics: five main types reported separately, GLOBAL split into normal vs NEW CUT.
- Every analytics query carries a `source` **filter** (ECITY / LEGACY / both, defaulting to both). With the businesses separated (§2.3) every row is ECITY and these figures describe the used-and-refurb business only — not the shop as a whole. Keep the dimension; it costs nothing and is what a later merge would need.
- Inventory movement `Opening → Purchases → Sales → Returns → Adjustments → Current` reconstructed from the stock ledger and device events.
- Drill-down routing Business → Branch → Transaction → Product/IMEI, terminating in the M9 device history page.

Built as:

- **One query layer, one shape.** A range, a set of branches, an optional comparison. Nine areas read from it rather than each inventing its own filtering — which is how two screens end up disagreeing about the same month.

- **No rollup tables, deliberately.** The plan suggested nightly per-branch-per-day rollups. At this shop's volume the live queries answer a twelve-month range in a fraction of the five-second budget, and a rollup would add a refresh to keep honest and a staleness window to explain. The functions would not change shape if it is ever needed.
- **The tests are hand-calculated, not read back from the code.** A dataset small enough to add up on paper — four bills across two branches — with every expectation worked out by hand: revenue ₹47,000, cost ₹30,200, gross ₹16,800, margin 35.74%. A test that asserts whatever the code returns proves only that the code is consistent with itself.
- **A branch-limited user asking for a branch they cannot see gets nothing, not everything.** The scope resolver returns a concrete list rather than "no filter" whenever the caller is limited — the failure mode of an empty filter meaning *unfiltered* is the one that leaks.
- **Margins are their own permission.** `analytics.view_profit` is separate from `analytics.view`, so a branch manager can be shown revenue without cost prices — the same line `inventory.view_cost` already drew.
- **Growth from nothing is null, not 100%.** There is no meaningful percentage change from a base of zero, and inventing one puts a number on screen that nobody can act on.
- **Charts are divs.** One bar per row against the largest value, no charting library: it works without JavaScript and costs nothing to ship. A library earns its place when a chart needs axes and interaction.
- **NEW CUT is a line inside GLOBAL on every table, and a filter only offered once GLOBAL is chosen** — the rule made visible, not merely enforced.
- **Business Insights (FR-35) is its own area**, because it answers a different question. Every other page says what happened; this one says what *changed* — revenue, average bill and margin against the period immediately before, which days of the week actually earn, and how the five main types compare. A figure on its own is a fact; a figure with a direction is a decision.
- **Best days are grouped by weekday, not by date.** "We are dead on Tuesdays" is something a shop can act on; "the 14th was slow" is not.
- **Margin change is in percentage points, not percent.** A margin going from 30% to 33% moved three points, not ten percent — the second reading is arithmetically defensible and completely useless.
- **A collection rate over 100% is correct, not a bug.** Money often arrives for bills raised before the window being looked at.
- **Every area has a chart and a table, and the comparison button is only offered where it means something.** Inventory and Credit lead with a *position* — stock on hand, what is owed now — which has no "compared to last month", so they do not show the control. A button that changes nothing is worse than no button.
- **The drill-through ends at the handset, not at the bill.** FR-36.4's path is Business → Branch → Transaction → Product/IMEI, and the last hop was missing: an invoice showed the IMEI as text. It is a link on screen and plain text in print, because paper has nowhere to click.
- **Found while building: the branch comparison sorted alphabetically.** `ORDER BY 2` points at whatever the second SELECT column happens to be, and here that was the branch *name* rather than revenue — so the owner's first question was answered in the wrong order. Every ordinal sort in the layer is now written as the expression it means, because the next person to add a column would have broken it again.
- **The source dimension reaches every table that can answer it.** `sale`, `purchase` and `device_unit` each carry a source, and one helper applies it to all three rather than nine call sites deciding for themselves. It filters nothing today — with the businesses separated (§2.3) every row is ECITY — but asking for LEGACY returns *nothing* rather than quietly ignoring the filter and returning everything, which is the failure mode that would matter after a merge.
- **Found while building: the movement report counted accessories only.** `stock_ledger` is non-serialised stock by design, so reconstructing FR-21's row from it alone described the smaller half of a phone shop — while "current" counted handsets too, which meant every device silently vanished into "opening". It now reads the

stock ledger *and* `device_event`, which is what the spec asked for: each event that moves a handset in or out of stock is ±1 unit, and events that change a device without moving it (inspected, reclassified, reserved, repaired) are deliberately absent.

- **Found while building: a backdated bill wrote its stock movement dated today.** `createSale` accepts `soldAt` but did not pass it to the stock ledger or the device event, so the movement report and the sales report described different days for the same bill. Both now carry the bill's date, and the same was true of a backdated purchase — its stock arrived on the wrong day, and a device could not be given an arrival date at all. `createDevice` now takes one, which M11's opening-balance import will need as well.

Done when.

- Every measure listed in PRD §6.15 exists and matches a hand-calculated result on the seeded test dataset.
- Switching the branch selector between one branch, two branches and all branches changes every figure correctly.
- Product and inventory analytics can show GLOBAL, and within GLOBAL split NEW CUT out separately.
- A 12-month analytics range for 10 branches returns in under 5 seconds.
- Clicking a dashboard number reaches an individual IMEI in at most four clicks.

Depends on. M7, M8. **Effort.** 3.0 weeks.

M11 — Reports, Exports, Imports & Opening Balances

Carried in — managing brands and categories (raised during M5). The API and permissions exist (`product.manage`) and the seed creates 15 brands and 13 categories, which is why the dropdowns work. There is no screen to add a sixteenth. Nobody noticed because the seeded lists were adequate; a real shop taking on a new brand hits a wall.

- **Brands** and **Categories** management screens (admin): create, rename, deactivate.
- Deactivate rather than delete — a category is referenced by products, and products by sales. Removing one would break invoices already issued.
- A category carries `isSerialised` and `identifierType`, which decide whether its products are tracked by IMEI, by serial, or by quantity. Those must be locked once any product uses the category: changing them would reinterpret existing stock.

On main types. `NEW / USED / ER / ACT / GLOBAL` stays a database enum, not a user-managed list. Main type is not a label — it decides which devices reach the till (FR-38.2), what the invoice prints, how GLOBAL carries NEW CUT, and how the dashboards group. A user-created sixth type would carry no behaviour and would silently sit outside all of those rules. Adding one is a migration and a deploy, and the question to answer first is what the new type *means*, because that is what determines the code.

Goal. Data gets in at the start and out whenever it is needed.

Delivers. FR-25.1 – FR-25.4, FR-33.1 – FR-33.3, FR-34.1 – FR-34.3.

Data model. `import_job`, `import_row_error`, `export_job`.

Screens. Report centre (sales, purchase, inventory, financial, credit, tax, reconciliation, branch comparison) with saved filters; export buttons producing CSV, Excel and PDF; import wizard — upload, map columns, validate, preview, commit — with a downloadable per-row error report; opening balance screens for inventory, cash/accounts and existing customer/supplier dues.

Server work. Streaming exports so large files do not exhaust memory; import validation that rejects a whole file on structural errors but reports row-level errors individually; imports carry branch, main type, GLOBAL/NEW CUT and

multiple IMEI columns per device row (accepted even while `imei_slots` is 1), and run through the same service layer as manual entry so that stock, ledger and device events stay consistent.

Built as:

- **One report shape, three formats.** A report returns columns and rows; the export layer turns that into CSV, Excel or PDF. Adding a report gets all three, and adding a format gets every report — rather than eight reports each describing themselves four times.
- **CSV streams, the other two buffer, and the PDF says so.** A spreadsheet and a PDF cannot be written without knowing where things end, so they are built in memory; the PDF refuses past 5,000 rows and names CSV instead, rather than quietly truncating. The CSV carries a byte-order mark so Excel opens it as UTF-8 instead of mangling any name with an accent.
- **Money goes into the spreadsheet as a number, not as text.** A spreadsheet that cannot sum its own money column is not much use. Converted from paise with integer arithmetic first, so no precision is invented.
- **Two phases, always: stage then apply.** A file is parsed into `import_row` on upload and nothing is created until someone has seen the preview. The wizard's state is in the database, so a browser that dies between mapping and committing loses nothing.
- **Structural problems reject the whole file; row problems are reported per row.** No header means nothing can be mapped, so there is nothing to salvage — importing "the rows that happened to parse" is how a shop ends up with silently partial stock. A bad row is named by its line number *as the spreadsheet shows it*, and the rest of the file still goes in.
- **The column mapping is guessed, and always correctable.** The guess knows the words people actually type — "IMEI No", "Cost Price", "Full Name" — but a guess that could not be overridden would be worse than none.
- **Imports run through the same services as manual entry.** The importer never writes to a table directly; that is how an import quietly produces records the rest of the system does not recognise. Stock, ledgers and device events come out identical to typing it in.
- **The same file twice is refused**, by content hash. Re-uploading is almost always a mistake, and importing it again would double everything in it.
- **CSV is parsed by hand; .xlsx is read with the library already here.** The CSV format is small and completely specified, and a dependency in the path that loads a shop's entire history is not worth it. A spreadsheet is another matter — and ExcelJS is already a dependency for writing exports, so reading the first sheet cost a function rather than a decision. Only the first sheet: a workbook with several is a question about which one, and guessing imports the wrong list. Numbers are stringified as digits, because an IMEI that arrives as `3.81E+14` matches nothing.
- **Two tables, not three.** The plan named `import_job`, `import_row_error` and `export_job`. A staged row and a rejected row are the same row at different moments — splitting them would mean writing every row twice and joining to find out which failed — so `import_row` carries the raw cells, the parsed values and the reason it could not be applied. And an export is a download that finishes inside the request; a job table would exist only to record that a file was made and then never read again.
- **Saved views keep the filters, never the figures.** A saved report re-runs against today's data; a stored result would go stale silently, which is the one thing a report must not do. Saving over a name you have used replaces it — adjusting a period and saving again is how this is actually used, and refusing would only teach people to write "sales 2".
- **Opening balances are typed in *or* filed, through one service either way.** A shop with three debtors types them; a shop with three hundred sends a spreadsheet. Both post through the same functions, so a figure looks identical whichever way it arrived and no screen has to know. A party named in a file is never created — a due against somebody the shop has not got is far more likely a spelling than a new account, and two people of the same name stop the row rather than guessing which one owes the money.
- **A stock file with no branch is refused on upload**, not one row at a time on the way in. Stock is held somewhere, and finding that out per row is the worst possible moment.

- **Found while building: changing a filter looked like it did nothing.** A filter change is a same-route navigation, so the App Router holds the URL — and the screen — until the server has rebuilt the report; on a shop's worth of history that is seconds of a page that appears not to have heard the click, which is how a report ends up being asked for three times. The controls now say "Rebuilding…" while it works. A route-level loading skeleton was tried first and taken back out: it fires on a filter change too, so the filters themselves vanished mid-interaction and came back as grey boxes — a page that answers a click by removing the thing that was clicked is worse than one that waits.
- **Opening balances post as movements, never as a balance column.** Every balance in the system is the sum of its movements (docs/03 §4.2); an opening figure that bypassed that would be the one number nobody could prove.
- **Found while building: customer dues could not see an opening balance.** The dues screen is built entirely from sales and ages each bill, so a balance carried in from before ECITY — which has no bill behind it — was invisible to the screen, the aging buckets, the dashboard and the credit report. Supplier dues sum the ledger directly and were already right; the asymmetry was only on the customer side.
- **Found while building: the Excel export hung forever.** ExcelJS waits on its output stream's own events; the hand-rolled sink I gave it satisfied TypeScript and then never fired them, so the endpoint returned nothing until the request timed out. A real `PassThrough` fixed it — and it is why the test asserts the file's first bytes rather than only a 200.
- **Carried in — brands and categories.** Built as specified. Nothing deletes: a category is referenced by products and products by invoices already issued. `isSerialised` and `identifierType` lock once the category has products, because changing them would reinterpret stock that already exists.

Done when.

- All seven report families produce correct output for a branch and consolidated, and export to CSV, Excel and PDF. *(Ten shipped: the seven named, branch comparison, and the customer and supplier lists FR-33.3 asks for by name.)*
- Inventory reports group the five main types with the GLOBAL/NEW CUT breakdown.
- Importing 1,000 devices with 20 deliberate errors imports 980, reports the 20 with row numbers and reasons, and leaves no partial rows; rows carrying two IMEIs import both.
- Opening balances produce correct starting stock, cash and customer/supplier outstanding, visible in the dashboards.

Depends on. M10. **Effort.** 2.0 weeks.

On OQ-9 — now closed. The importer is mapping-driven, so it never needed the shop's file format in advance; that is what unblocked this module. What remained was the order, which the service layer decides for us — an import can only reference what already exists — so the go-live sequence, its checks, and what is deliberately left behind are written up in **docs/04 §10**.

M12 — External Billing System Integration (Excel Feed) — DEFERRED

> **Not being built for v1.0** (decided 2026-09-07, see §2.3). The two businesses are separated: NEW stock lives entirely in the other system, ECITY handles everything else, and nothing moves between them. This module existed to reconcile two systems sharing the *same* stock, which is no longer the arrangement.

>

> The design below is kept, unchanged and unstarted. The hooks it needs are already in the schema, so if the shop ever wants NEW sales visible in here too, the work starts from this page rather than a blank one.

Goal. The shop keeps using its existing billing system for NEW items. This module makes that survivable: their Excel export becomes a controlled daily feed into ECITY, stock stays truthful, and the same IMEI can never be sold twice.

Delivers. FR-38.1 – FR-38.14.

The problem this solves

Two systems touch the same physical stock. The legacy system bills NEW items; ECITY bills everything else and owns inventory, credit, cash and reporting. Without a design, three things break:

1. **Double sale.** The legacy system sells IMEI X at 11 AM. ECITY still shows it In Stock. At 4 PM someone sells it again in ECITY. Two invoices, one phone.
2. **Cash never tallies.** Legacy sales take real cash into the same physical drawer. ECITY's expected cash is short by exactly that amount until the file is uploaded, so daily closing is meaningless.
3. **Reports lie.** Sales, profit and stock figures are missing everything the other system did.

The design: block by channel, do not race the clock

The instinct is to mark stock as sold and let the nightly upload confirm it. That works, but it still leaves a window in which ECITY believes a device is sellable. **Close the window instead of shrinking it.**

Every device carries a `sales_channel`: `ECITY`, `EXTERNAL`, or `BOTH`, defaulted from `main_type` (NEW → `EXTERNAL`, everything else → `ECITY`) and overridable per device.

- The ECITY billing screen **refuses** to sell a device whose channel is `EXTERNAL`, with a plain message: *"This device is billed through the other system."*
- So ECITY's stock is never *dangerously* wrong. It may be a few hours stale — it still shows the device as held — but nobody can double-sell it.
- The daily upload converts those devices to `Sold`, attaching the real invoice number, date, customer and price.

For anything sold in ECITY that the legacy system also handles (`BOTH`), a manual action **Mark as sold externally** sets the status `SOLD_PENDING_IMPORT`. Stock drops immediately, and the upload later completes the record with the real invoice. This is the fallback path, not the main one.

The Excel format is unknown — build everything except the last 100 lines

The file layout cannot be designed for yet. **Structure the code so that only one small file needs writing when it arrives:**

```
/server/services/external-import/
  types.ts              <- canonical row shapes. STABLE. Write this first.
  adapters/
    legacy-sales.adapter.ts      <-- TODO: the ONLY file that knows the
    legacy-purchase.adapter.ts   <-- real Excel column layout
  mapping/
    legacy.mapping.json <- column-name -> canonical-field map, editable
                           without a code change or redeploy
  ingest.ts             <- upload, parse, stage, validate, dedupe
  match.ts              <- resolve IMEI/product/customer to our records
  apply.ts              <- commit via the SAME services as manual entry
  reconcile.ts          <- daily comparison + exception report
```

Everything downstream of `types.ts` is written against the canonical shape and can be built **now**, before anyone has seen the file:

```
// types.ts - the contract. Does not change when the Excel changes.
type ExternalSaleRow = {
  externalInvoiceNo: string
  externalLineId:    string        // for idempotency
  soldAt:            Date
```

```
  imei?:             string        // mobiles
  sku?:              string        // accessories
  qty:               number
  unitPrice:         bigint        // paise
  discount:          bigint
  tax:               bigint
  paymentMethod?:    string
  customerName?:     string
  customerPhone?:    string
  branchCode?:       string
  raw:               Record<string, unknown>   // the untouched source row
}
```

When the real file arrives, the work is: read the columns, fill in `legacy.mapping.json`, write ~100 lines in the adapter, add fixture tests from a real export. **Two to three days**, not a redesign. Prefer the JSON mapping over hard-coded column names — the legacy vendor will change a header eventually, and that should be a config edit, not a release.

Data model. `import_batch` (file, hash, uploaded_by, period, status, counts), `import_row` (batch, row number, raw payload, canonical payload, match result, applied ref, error), `external_reference` (our entity ↔ their invoice/line id, unique), and on existing tables: `source` (`ECITY` | `LEGACY`) plus `sales_channel` and `SOLD_PENDING_IMPORT` on `device_unit`.

Screens. Upload page with drag-and-drop and batch history. The wizard's step state is server-backed (`import_batch` / `import_row`), so it needs no client store — a browser crash mid-review must not lose a staged batch; **preview before commit** showing what will be created, matched, skipped and rejected, with per-row reasons; exception queue for unmatched rows with resolve actions (link to an existing device, create it, ignore with a reason); daily reconciliation report; a "today's feed not yet uploaded" banner on the dashboard.

Server work.

- **Idempotency.** Key every row on `(source, externalInvoiceNo, externalLineId)` and store the file hash. Re-uploading the same file changes nothing; re-uploading a corrected file updates only what actually differs.
- **Two-phase: stage then apply.** Parse and validate into `import_row` first, show the operator the preview, and only commit on confirm. Nothing is half-applied — the whole batch commits or none of it does.
- **Apply through the existing service layer.** Imported sales call the same `createSale()` / `sellDevice()` functions as the counter, so stock, ledgers, `device_event` rows and analytics stay consistent by construction. Never write directly to tables from the importer.
- **Unmatched rows.** A sale of an unknown IMEI is *not* auto-created — it goes to the exception queue, because selling something we never bought is a data gap worth a human look. Purchase rows for unknown IMEIs *do* auto-create the device with `source = LEGACY`.
- **Reversal.** A whole batch can be reversed, producing reversal documents rather than deletions.
- **Daily closing gate (M7).** A branch's day cannot be closed until today's legacy file is uploaded, or an authorised user records an explicit override with a reason. Otherwise expected cash is guaranteed to be wrong and the mismatch figure is noise.

Done when.

- Uploading the same file twice produces zero duplicate sales.
- A NEW device cannot be sold on the ECITY billing screen; the message explains why.
- A file with 500 rows, of which 20 are unmatched IMEIs and 5 malformed, previews accurately, commits the 475, and queues the 25 with reasons.
- After committing a sales file, stock, customer dues, cash figures, profit and the IMEI device history all reflect the legacy sales identically to how they would if typed in by hand.

- The device history of a legacy-sold phone shows the sale with its real invoice number and is labelled as coming from the other system.
- A branch cannot close its day with today's file missing, unless overridden with a reason that appears in the audit log.
- Reversing a batch cleanly restores stock and balances.

Depends on. M7 (cash and closing) and M11 (shares the import infrastructure). **Effort.** 3.5 weeks — of which **3.0 is format-independent and can start immediately**, and 0.5 is the adapter once a real export is in hand. *Blocked on PRD OQ-10 and OQ-11.*

M13 — Notifications, Alerts & Warranty

Goal. The system tells you what needs attention instead of waiting to be asked.

Delivers. FR-27.1 – FR-27.3, FR-29.1, FR-29.2.

Data model. `notification`, `notification_rule`, warranty fields on `device_unit`.

Screens. Notification centre with unread state and per-user preferences; alert cards on the dashboards; warranty view on device history; warranty expiry list.

Server work. A scheduled job evaluating the rules — low stock, overdue customer payments, supplier dues, cash mismatch, unclosed day, stock adjustment, warranty expiry — writing notifications scoped to branch users, with consolidated versions for owners/admins. Deduplicate so the same condition does not notify daily forever.

Done when.

- Each of the seven triggers fires on a constructed test condition and reaches exactly the right users.
- A branch user sees only their branch's alerts; the owner sees all, grouped by branch.
- Warranty details appear on the device history page and an expiring-soon list is available.

Depends on. M7. **Effort.** 1.5 weeks. *Blocked on PRD OQ-6 if external channels are wanted.*

M14 — Backup, Data Protection, Hardening & Go-Live

Goal. The system is safe to trust with a real business.

Delivers. FR-31.2, FR-31.3, FR-32.1 – FR-32.3, and PRD §9 non-functional requirements.

Work.

- Three backup layers per the Deployment Guide §7: Hetzner snapshots, `pg_dump` to Cloudflare R2 via restic (7 daily / 4 weekly / 6 monthly), and a **restore actually performed** into a scratch database, timed, with the procedure written down.
- **Decide the acceptable data-loss window.** Nightly dumps alone mean losing up to a full trading day. Two ways to close that: dump every four hours (one crontab line, ~4-hour worst case), or add WAL archiving with `pgBackRest`/`wal-g` to R2 for true point-in-time recovery (~5-minute worst case, about half a day of setup). **Recommended: both** — this is money data.
- Owner-triggered full business data export.
- Soft-delete and reversal states audited across all financial and inventory documents; confirm no destructive path remains.

- Security pass: authorisation tests on every endpoint for role × branch, rate limiting on login and search, signed URLs on all uploads, dependency audit, secret handling.
- Performance pass against the PRD §9.1 targets on a dataset the size of the §9.1 sizing assumption; add the indexes the traces demand.
- Go-live: production environment, monitoring and error tracking, uptime alerting, opening balances loaded, staff training, and a two-week parallel-run period before paper records are retired. Staff training must cover the **daily external-feed upload** — it is a new habit the shop has to form, and everything downstream of it (stock, cash, closing, reports) is wrong on any day it is skipped.

Done when.

- A restore from backup into a clean environment is demonstrated and timed, and the chosen data-loss window is met in that test.
- An authorisation test suite covers every endpoint for each role and branch combination and passes.
- Performance targets are met on full-size data.
- Branch one has run live in parallel for two weeks with no unexplained discrepancy in cash, stock or dues.

Depends on. All. **Effort.** 1.5 weeks.

3. Suggested First Two Weeks

1. Answer PRD OQ-1, OQ-2, OQ-3, OQ-7 and OQ-11 with the shop owner — these change the schema or the sales module, and are cheap to answer now and expensive to answer later. OQ-10 (a real Excel export) is parked with M12 (§2.3); ask for the sample only if the shop asks for the integration.
2. Set up the repository, database, deployment and CI (start of M0).
3. Write the full schema for M0–M2 up front, even though you build it in stages, and include `device_identifier` as a separate table from the very first migration even though the UI will show a single IMEI field. The device, identifier and event tables are load-bearing for every later module, and rewriting them after M4 is the single most expensive mistake available in this project.